Project Documentation : tls_forcer

- Protyasha Kundu

Overview

tls-forcer is a C++ toolkit that runs real TLS handshakes with websites such as google.com, forces the client's choice of algorithms, and shows every message, key and timing live in Chrome.

The key idea: in TLS the **client only offers a menu and the server chooses**. The client can force an algorithm by putting just one item on the menu, so the server must either use it or refuse.

The project has four programs: a live dashboard, a client, a prober that maps what a server accepts, and a local test server. All are built on OpenSSL 3, and post-quantum algorithms are available with OpenSSL 3.5.

What is TLS?

TLS (Transport Layer Security) is the protocol that protects connections on the internet. It is what the padlock in your browser means, and the "S" in HTTPS.

When a machine talks to a website, the messages pass through many other computers on the way. TLS gives the connection three guarantees:

Goal	How TLS achieves it
Confidentiality	A shared secret key and a symmetric cipher (e.g. AES-GCM)
Authentication	A certificate (ID card) signed by a trusted authority, plus a signature from the server
Integrity	Every record carries a check value; the handshake ends with Finished messages

Before any data flows, the two sides run a short conversation called the **handshake**. In it they agree on algorithms, create a shared secret without ever sending it, and check the server's identity. **TLS 1.3** (RFC 8446, 2018) is the current version: it needs only one round trip and encrypts most of the handshake. **TLS 1.2** is the older version, still widely supported.

TLS is used far beyond web pages: email, APIs, VPNs, messaging apps, and DNS over TLS all rely on it.

The TLS 1.3 handshake, step by step

A TLS 1.3 handshake takes one round trip: the client speaks, the server answers with everything at once, and the client confirms. Think of ordering at a restaurant: you hand over a menu, the kitchen picks a dish and shows its licence, and both sides confirm the order.

1. **ClientHello** (client → server, plaintext): "Here is my menu." It lists TLS versions, cipher suites, key-exchange groups and signature schemes, names the website it wants (SNI), and includes a **key share**: the client's public half of the key exchange, made in advance for its first-choice group.
2. **ServerHello** (server → client, plaintext): "I pick this one, and here is my half." The server chooses from the menu and sends its own key share. Both sides now combine the two halves into the **same shared secret** without sending it. Everything after this point is encrypted.
3. **EncryptedExtensions** (encrypted): extra settings the server chose, such as the web protocol (HTTP/2).

4. **Certificate** (encrypted): the server's ID card: its public key and the site names it covers, signed by a trusted Certificate Authority.
5. **CertificateVerify** (encrypted): the server signs a hash of the whole conversation with its certificate's privte key. This proves it owns the certificate and that nobody changed the client's menu.
6. **Finished** (server, encrypted): a check value over the entire handshake.
7. **Finished** (client, encrypted): the client's own check value. The handshake is complete and application data can flow.
8. **NewSessionTicket** (after the handshake): a ticket for a faster reconnection next time.

Two special cases appear in the project:

- **HelloRetryRequest**: if the server doesn't accept the client's guessed key share but likes another group on the menu, it asks the client to try again. This costs one extra round trip.
- **Alert: handshake_failure**: nothing on the menu was acceptable, so the server ends the connection.

TLS 1.2 differs: it needs two round trips, sends the certificate in the clear, and exchanges key shares in separate ServerKeyExchange and ClientKeyExchange messages.

What can be forced

Inside TLS 1.3, three choices are negotiated separately, and the project forces each one by sending a one-item list in its own ClientHello field.

What	Its job	ClientHello field	Examples
Key exchange (group)	Makes the shared secret	`supported_groups` + `key_share`	X25519, P-256, P-384, X25519MLKEM768 (post-quantum)
Signature scheme	Proves the server's identity (CertificateVerify)	`signature_algorithms`	ecdsa_secp256r1_sha256, rsa_pss_rsae_sha256, ed25519, mldsa65 (post-quantum)
Cipher suite	Encrypts the data once the key exists	`cipher_suites`	TLS_AES_128_GCM_SHA256, TLS_AES_256_GCM_SHA384, TLS_CHACHA20_POLY1305_SHA256
Protocol version	Which rulebook to use	`supported_versions`	TLS 1.3, TLS 1.2

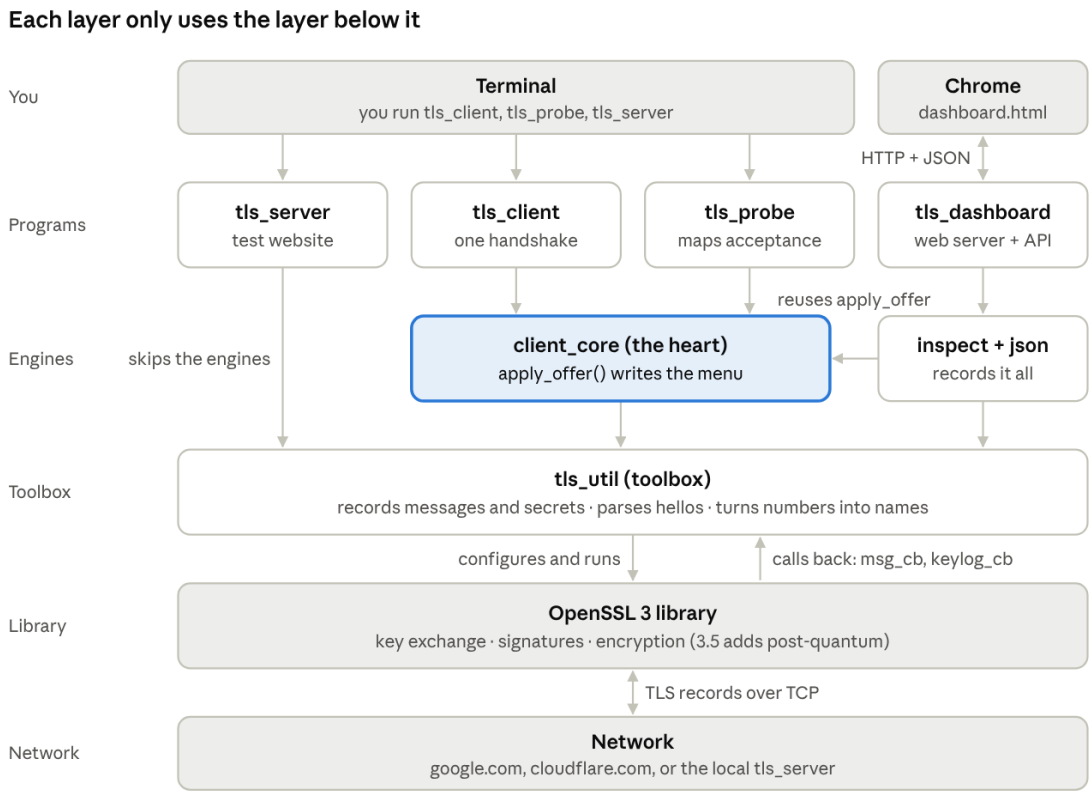
In TLS 1.2 these were bundled into one cipher-suite name (for example `ECDHE-ECDSA-AES128-GCM-SHA256`), so TLS 1.3 makes forcing much more precise.

What forcing can do: stop the server from picking anything except your choice, and override the server's own preference order.

TLS 1.3 itself has no defaults, only a mandatory minimum that every implementation must support (AES-128-GCM, P-256, and RSA or ECDSA with SHA-256). The defaults the client offers come from OpenSSL, and the fiinal pick comes from the server's own preference.

Project architecture

The project is built in layers: our code writes the menu and watches the handshake, and OpenSSL does all the cryptography.



tls-forcer architecture · 6 layers

Shaded boxes are outside pieces; the highlighted box is where forcing happens. `tls_server` skips the engines because a server reads a menu instead of writing one.

One click, end to end:

1. click **Force P-256** in Chrome; the page sends `GET /api/handshake?host=google.com&groups=P-256` to `tls_dashboard`.
2. `tls_dashboard` builds a `ClientConfig` and calls `inspect()`.
3. `inspect()` calls `apply_offer()`, which puts only P-256 on the menu.
4. `attach_trace()` registers the callbacks, then `SSL_connect()` makes OpenSSL run the handshake with Google.
5. OpenSSL calls `msg_cb` for every message and `keylog_cb` for every secret.
6. `to_json()` labels each message encrypted or plaintext and packs everything as JSON.
7. The page draws the arrows, details, timing, certificate chain and keys.

What each file does

Every source file has one primary role. The engines and toolbox are compiled once into a shared library (`tlsf`) that all four programs use.

File	Layer	Primary role	Key functions
`src/`	Engine	**The menu writer.** Turns your choices	`apply_offer()`,

File	Layer	Primary role	Key functions
`client_core.hpp/.cpp`		into the ClientHello (the forcing) and runs one handshske	`run_handshake()`, `ClientConfig`, `HandshakeResult`
`src/inspect.hpp/.cpp`	Engine	**The reporter with a stopwatch.** One fully recorded handshake for the dashboard: timings, messages, certificates, keys, encrypted/plaintext labels	`inspect()`, `to_json()`, `cert_info()`, `connect_with_timeout()`
`src/tls_util.hpp/.cpp`	Toolbox	**The note-taker and translator.** Records every message and secret, parses hellos byte by byte, spots retries, turns numbers into names	`attach_trace()`, `msg_cb()`, `keylog_cb()`, `parse_hello()`, `describe()`, `*_name()`, `is_grease()`
`src/json.hpp`	Toolbox	**The packer.** Writes JSON for the browser without an external library	`str()`, `num()`, `arr()`, `obj()`
`src/tls_client.cpp`	Program	**The messenger.** One forced handshake from the terminal; `--keys` prints the key material	`main()`
`src/tls_probe.cpp`	Program	**The taste tester.** Offers each option alone to map what a server accepts, and tests whose preference wins	`probe_axis()`, `preference_test()`
`src/tls_server.cpp`	Program	**My own test website.** Logs what each client offered next to what it chose; works with browsers too	`client_hello_cb()`, `handle()`, `report()`
`src/tls_dashboard.cpp`	Program	**The waiter.** Local web server on 127.0.0.1:8088 that runs handshakes for the page	`api_info()`, `api_handshake()`, `api_probe()`, `handle()`
`web/dashboard.html`	Page	Calls the API and draws animated arrows, details and keys	`run()`, `buildSeq()`, `detail()`, `keysForMessage()`, `renderKeys()`, `probe()`
`CMakeLists.txt`	Build	Builds the library and four programs, and packs the page into the dashboard	`file(READ)`, `configure_file()`
`src/dashboard_html.hpp.in`	Build	**Packing template.** CMake fills it with the page to make `build/generated/dashboard_html.hpp`	`kDashboardHtml`
`scripts/`	Setup	**Fake-ID maker.** A test CA plus ECDSA,	`leaf()`

File	Layer	Primary role	Key functions
`gen_certs.sh`		Ed25519, RSA (and ML-DSA on OpenSSL 3.5) certificates	

How to build and run

The project needs a C++17 compiler, CMake and OpenSSL 3. Post-quantum algorithms (ML-KEM, ML-DSA) need OpenSSL 3,5, which can be built into `~/ossl35` without touching the system copy.

Build (from the folder containing `CMakeLists.txt`):

```
sudo apt install build-essential cmake libssl-dev
rm -rf build
cmake -S . -B build -DOPENSSL_ROOT_DIR=$HOME/ossl35   # drop the flag for system
OpenSSL 3.0
cmake --build build -j
./scripts/gen_certs.sh certs                          # test certificates for
tls_server
```

Run:

Program	Command	What you get
Dashboard	`./build/tls_dashboard`, then open http://localhost:8088	Live arrows, details, keys, probes, history
One forced handshake	`./build/tls_client --host google.com --group P-256 --keys`	What the server chose, plus the keys
Map a server	`./build/tls_probe --host google.com`	YES / NO per algorithm, and who decides
Test server	`./build/tls_server --port 4433 --cert certs/ecdsa.crt --key certs/ecdsa.key`	"Client offered / server chose" for every connection

With the OpenSSL 3.5 build, run `export LD_LIBRARY_PATH=$HOME/ossl35/lib` once in each terminal first. To decrypt a capture in Wireshark, save the secrets with `--keylog keys.txt` or the dashboard's **Save key log** button, then load the file under TLS preferences.

Running on a remote machine

The project runs same way on a remote Linux server (a lab machine, cloud VM or Raspberry Pi) as on a laptop the only differences are getting the code there and reaching the dashboard, which listens on the remote machine's localhost only.

1. Log in and install the tools (Ubuntu or Debian):

```
ssh user@remote-host
sudo apt update && sudo apt install build-essential cmake libssl-dev git
```

2. Get the code (from GitHub, or copy it from your laptop):

```
git clone https://github.com/ProtyashaKundu/TLS-FORCER.git tls-forcer
# or, from the laptop:  rsync -av --exclude build --exclude certs ./tls-forcer/
user@remote-host:~/tls-forcer/
cd tls-forcer
```

3. Optional: OpenSSL 3.5 for post-quantum (about 10 minutes, installed into your home folder):

```
git clone --depth 1 -b openssl-3.5 https://github.com/openssl/openssl.git ~/openssl-3.5-src
cd ~/openssl-3.5-src && ./Configure --prefix=$HOME/ossl35 --libdir=lib && make -j$(nproc) && make install_sw
cd ~/tls-forcer
```

4. Build and make test certificates:

```
rm -rf build
cmake -S . -B build -DOPENSSL_ROOT_DIR=$HOME/ossl35   # drop the flag to use the system OpenSSL
cmake --build build -j
./scripts/gen_certs.sh certs
export LD_LIBRARY_PATH=$HOME/ossl35/lib               # only for the OpenSSL 3.5 build
```

5. Command-line tools work directly over SSH:

```
./build/tls_client --host google.com --group X25519MLKEM768 --keys
./build/tls_probe --host cloudflare.com
```

6. Open the dashboard from your laptop through an SSH tunnel. The dashboard deliberately listens on `127.0.0.1` only, so it can't be reached directly over the network. Forward the port instead:

```
# on the remote machine (tmux keeps it running after you log out)
tmux new -s tls
./build/tls_dashboard
# detach with Ctrl+B then D

# on your laptop, in a new terminal
ssh -N -L 8088:localhost:8088 user@remote-host
```

Then open http://localhost:8088 in Chrome on your laptop. Every handshake now runs from the remote machine, so timings and results reflect its network, not yours.

7. The test server on a remote machine. `tls_server` listens on all network interfaces. The test certificates are only valid for `localhost`, so either reach it through a tunnel (`ssh -N -L 4433:localhost:4433 user@remote-host`, then use `--host localhost --port 4433`), or add the remote machine's name to `subjectAltName` in `scripts/gen_certs.sh` and regenerate the certificates. If you open port 4433 in the firewall (`sudo ufw allow 4433/tcp`), close it again afterwards.

8. Copy results back (key logs, probe output):

```
scp user@remote-host:~/tls-forcer/keys.txt .
```

Never expose the dashboard port to the internet: it has no login, and anyone reaching it could make the machine run handshakes to any site. The SSH tunnel keeps it private.

Key findings

Running the tools against real servers shows that forcing works only as a veto, and that servers following the same TLS 1.3 rules still behave differently.

- **Forcing is a veto.** Forcing P-256 or ChaCha20 on google.com succeeds; forcing Ed25519 or Ed448 signatures fails with `handshake_failure`, because Google's certificates are ECDSA or RSA.
- **Two kinds of failure.** "Not sent" means the local OpenSSL refused the name (for example ML-KEM on OpenSSL 3.0), so nothing left the laptop "Refused" means the server answered with an alert.
- **HelloRetryRequest on demand.** Offering `ffdhe2048:X25519` makes OpenSSL send a key share for ffdhe2048 only; Google rejects it, asks for X25519, and the handshake takes two round trips The retry is recognised by its fixed "random" value, SHA-256("HelloRetryRequest").

- **Same rules, different timing.** For an impossible signature, Google refuses straight after the ClientHello, in plaintext. Cloudflare first agrees on keys and sends its 4 KB certificate, then aborts with an encrypted alert. TLS 1.3 fixes the outcome, not the moment of the check.
- **Keys made visible.** The dashboard shows both randoms, both public key shares and every derived secret, and which secret encrypted each message. A refusal after the key exchange leaves handshake secrets but no data secrets.
- **Post-quantum, half way.** With OpenSSL 3.5, Google and Cloudflare accept the hybrid key exchange X25519MLKEM768. The client key share grows from 32 to about 1,216 bytes, and the ClientHello to about 1,387 bytes, Post-quantum signatures (ML-DSA) fail on public sites, because no public CA issues ML-DSA certificats yet; they work only against the local test server.

Scope and future work

The same idea, restricting the offer and watching the negotiation, extends to other secure protocols and to everyday browsing.

Other protocols

Ping (ICMP) has no cryptography, so there is nothing to force; its role is as a **network baseline**. Comparing ping's round-trip time with the TCP connect time and the TLS handshake time separates network delay from cryptographic cost.

Protocol	Uses	What could be forced and observed
ping / traceroute (ICMP)	Reachability, latency	Round-trip baseline to compare with handshake time; the route taken
SSH	Remote login	Key exchange (including post-quantum sntrup761 / mlkem768), host-key and cipher algorithms
QUIC / HTTP/3	Modern web over UDP	TLS 1.3 runs inside QUIC; same groups, signatures and suites
DTLS	TLS over UDP (VPNs, WebRTC)	The same three choices as TLS
IKEv2 / IPsec	VPN tunnels	Diffie-Hellman groups, encryption and integrity algorithms
WireGuard	VPN	Fixed algorithms (no negotiation): observe only, a useful contrast
DNS over TLS / HTTPS	Private DNS	Plain TLS 1.3 to resolvers such as 1.1.1.1

Capturing cryptographic details while browsing

The dashboard shows handshakes the tool makes itself. Capturing the browser's own handshakes during normal browsing is the natural next step, and three approaches fit:

1. **Key log from the browser.** Start Chrome or Firefox with the `SSLKEYLOGFILE` environment variable set; the browser writes every session's secrets, and Wireshark decrypts the captured traffic. This is the same format the project already produces.

2. **Passive handshake sniffer.** A small program (for example with libpcap) reads every ClientHello and ServerHello on the network card and logs the site name (SNI), offered and chosen groups, suites, key-share sizes and retries, reusing `parse_hello()`. It doesn't need key, because hellos travel in the clear. In TLS 1.3 the certificate is encrypted unless a key log is also used.
3. **Local test server.** Pointing a browser at `tls_server` already show the browser's real menu, including GREASE values and post-quantum key shares.

Limits to plan for: Chrome often uses QUIC (UDP) for Google sites, so a TCP-only sniffer misses those; Encrypted Client Hello (ECH) hides the site name; and capturing should cover only your own traffic.

Glossary

Term	Meaning in simple words
Client / server	Your computer (the visitor) / the website's computer
Handshake	The opening conversation where both sides agree on algorithms and a secret key
ClientHello / ServerHello	The client's menu / the server's choice
Key exchange (group)	The method that creates the shared secret, e.g. X25519, P-256
Key share	One side's public half of the key exchange
Signature scheme	How the server signs to prove its certificate is its own
Cipher suite	The method that encrypts data afterwards, e.g. AES-256-GCM
Certificate	The server's ID card, sign by a Certificate Authority (CA)
SNI	The website name the client asks for in its ClientHello
HelloRetryRequest	The server asking the client to resend its key share for another group
Alert / handshake_failure	The message that ends a failed handshake
Traffic secrets	The secrets both sides derive; the actual encryption keys come from them
Key log (SSLKEYLOGFILE)	A text file of session secrets that lets Wireshark decrypt a capture
GREASE	Fake values browsers send on purpose so servers that break on unknown values are caught early
IANA	The authority that assigns the official number for every TLS algorithm
NID	OpenSSL's own internal number for an algorithm
secp256r1 / P-256 / prime256v1	Three names for the same elliptic curve

Term	Meaning in simple words
FFDHE	Classic Diffie- Hellman over a large prime; large and slow, rarely used in TLS 1.3
ML-KEM / ML-DSA	Post-quantum key exchange / post-quantum signatures
X25519MLKEM768	Hybrid key exchange: classic X25519 plus post-quantum ML-KEM-768
OpenSSL	The library that performs all the cryptography; the project controls and observes it

References

TLS standards (IETF)

- RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3. The TLS 1.3 handshake, HelloRetryRequest, key schedule and mandatory algorithms (§9.1).
- RFC 5246: The TLS Protocol Version 1.2. The older handshake that the `--tls12` runs use.
- RFC 6066: TLS Extensions: Extension Definitions. Server Name Indication (SNI).
- RFC 7919: Negotiated Finite Field Diffie-Hellman Ephemeral Parameters for TLS. The ffdhe2048 to ffdhe8192 groups.
- RFC 8701: Applying GREASE to TLS Extensibility. The fake values that browsers add to catch strict servers.
- IANA Transport Layer Security (TLS) Parameters. The official number for every group, signature, cipher suite and extension; the name tables in `tls_util.cpp` come from here.

Post-quantum cryptography

- Post-quantum hybrid ECDHE-MLKEM Key Agreement for TLSv1.3 (IETF draft). Defines X25519MLKEM768 (0x11EC).
- NIST FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM).
- NIST FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA).
- Open Quantum Safe: oqs-provider. PQ algorithms for OpenSSL versions before 3.5.

OpenSSL documentation (3.5)

- SSL_CTX_set1_groups_list / SSL_CTX_set1_curves. Sets the key-exchange menu.
- SSL_CTX_set1_sigalgs_list. Sets the signature menu.
- SSL_CTX_set_cipher_list / SSL_CTX_set_ciphersuites. Sets the cipher-suite menu.
- SSL_CTX_set_msg_callback. The trace hook (`msg_cb`).
- SSL_CTX_set_keylog_callback. The key-log hook behind the Keys card.
- SSL_CTX_set_client_hello_cb. Used by `tls_server` to read what the client offered.

Decrypting with Wireshark

- NSS Key Log Format. The `CLIENT_RANDOM` / `*_TRAFFIC_SECRET` line format that `--keylog` writes.
- Wireshark wiki: TLS. Loading a key log file to decrypt a capture.